

Guía de Buenas Prácticas para Trabajar con Git y GitHub en Equipo

1. Estructura de Ramas

- **main**: Rama final con código aprobado, limpio y funcional.
 - **developer**: Rama de integración donde se fusiona el trabajo de todos antes de pasarlo a main.
 - **sabrina / emanuel**: Ramas personales de desarrollo.
-

2. Flujo Diario de Trabajo

Al comenzar a trabajar:

```
git checkout TU_RAMA
```

```
git checkout sabrina
```

```
git checkout emanuel
```

```
git pull origin developer
```

```
# trae los últimos cambios que hizo tu compañero
```

Hacer tus cambios:

```
# Trabajá en tu código normalmente
```

```
# Luego:
```

```
git add .
```

```
git commit -m "Descripción de lo que hiciste"
```

```
git push origin TU_RAMA
```

3. Unir tu trabajo en la rama developer

Una vez terminado un bloque de trabajo:

```
git checkout developer
```

```
git pull origin developer
```

```
git merge TU_RAMA
```

```
git push origin developer
```

LUEGO PARA VOLVER A LA RAMA PROPIA

```
git checkout sabrina // git checkout emanuel
```

```
git pull origin developer
```

4. Unir developer en main (solo si todo está probado)

```
git checkout main
```

```
git pull origin main
```

```
git merge developer
```

```
git push origin main (AL ÚLTIMO)
```

5. Buenas Prácticas Generales

- Nunca trabajar directo sobre `main`
- Siempre hacer `git pull` antes de empezar
- Evitar conflictos trabajando en archivos distintos o comunicándose bien
- Usar mensajes de commit claros y concretos
- No subir `node_modules/` ni archivos pesados
- Si hay conflictos al hacer merge, resolverlos con cuidado y luego:

```
git add .
```

```
git commit -m "Resuelvo conflictos"
```

6. Crear una nueva rama desde main o developer

```
git checkout main # o developer  
git pull origin main # o developer  
git checkout -b nueva-rama
```

7. Verificar el estado de Git

```
git status
```

8. Recuperar trabajo si algo sale mal

```
git stash # guarda los cambios sin commitear  
git stash pop # los recupera
```